

```
#include <ServoTimer2.h>

// Motor A (RIGHT) connections
int enA = 3;
int in1 = 4;
int in2 = 5;

// Motor B (LEFT) connections
int enB = 8;
int in3 = 6;
int in4 = 7;

//Payload connection
int servoPayloadPin = 9;
int payloadAngle = 0;
ServoTimer2 payloadServo;

//Ultrasonic connection
int distance_in = 0;
int trigPin = A1;
int echoPin = A0;
int servoUltrasonicPin = 9;
int ultrasonicAngle = 0;

ServoTimer2 ultrasonicServo;

#define NUM_READINGS 3

int distanceReadings[NUM_READINGS]; // array to store last 10 values
int readIndex = 0;           // current position in array
```

```
long total = 0;           // sum of readings
int averageDistance = 0;   // final average
```

```
//Pin Assignments and Servo Attachment
```

```
void setup() {
```

```
    //Serial Monitor for debugging
```

```
    Serial.begin (9600);
```

```
    // Set all the motor control pins to outputs
```

```
    pinMode(enA, OUTPUT);
```

```
    pinMode(enB, OUTPUT);
```

```
    pinMode(in1, OUTPUT);
```

```
    pinMode(in2, OUTPUT);
```

```
    pinMode(in3, OUTPUT);
```

```
    pinMode(in4, OUTPUT);
```

```
// Turn off motors - Initial state
```

```
digitalWrite(in1, LOW);
```

```
digitalWrite(in2, LOW);
```

```
digitalWrite(in3, LOW);
```

```
digitalWrite(in4, LOW);
```

```
// Payload - Initial State ON
```

```
digitalWrite(servoPayloadPin, HIGH);
```

```
// Attach payload servo to the pin
```

```
//Ultrasonic pin setup
```

```
pinMode(trigPin, OUTPUT);
```

```
pinMode(echoPin, INPUT);
```

```
// Attach ultrasonic servo to the pin
```

```
ultrasonicServo.attach(servoUltrasonicPin);
```

```
for (int i = 0; i < NUM_READINGS; i++) {
```

```
    distanceReadings[i] = 0;
```

```
}
```

```
}
```

```
//Main program execution
```

```
void loop() {
```

```
    driveContinuously();
```

```
    unsigned long startTime = millis();
```

```
    while (millis() - startTime < 50) {
```

```
        turnUltrasonicServo();
```

```
        stopDriving();
```

```
        driveBackwards();
```

```
        turnLeft();
```

```
}
```

```
distance_in = 100;
```

```
}
```

```
//Function for ultrasonic scan and echo return
```

```
void scanForObstacles(){
```

```
    digitalWrite(trigPin, HIGH);
```

```
    delayMicroseconds(10);
```

```
    digitalWrite(trigPin, LOW);
```

```
    int duration_us = pulseIn(echoPin, HIGH);
```

```
    int rawDistance = 0.0067 * duration_us;
```

```
    distance_in = getAverageDistance(rawDistance);
```

```
    Serial.print("distance: ");
```

```
    Serial.print(distance_in);
```

```
    Serial.println(" in");
```

```
    unsigned long startTime = millis();
```

```
    while (millis() - startTime < 10) {
```

```
        // Do something repeatedly for 2 seconds
```

```
}
```

```
}
```

```

//Function for continous sweep and scan of ultrasonic sensor
void turnUltrasonicServo(){
    for(ultrasonicAngle = 1500; ultrasonicAngle <= 1500; ultrasonicAngle += 500) {
        ultrasonicServo.write(ultrasonicAngle); // Set the servo position

        Serial.print("Angle: ");
        Serial.println(ultrasonicAngle);
        unsigned long startTime = millis();
        delay(100);
        while (millis() - startTime < 2000) {
            scanForObstacles();
            if(distance_in < 18){
                return 0;
            }
        }
        // Wait 5ms for the servo to reach the position
    }

    // Sweep back from 90 to 0 degrees
    for(ultrasonicAngle = 1500; ultrasonicAngle >= 1500; ultrasonicAngle -= 500) {
        ultrasonicServo.write(ultrasonicAngle);

        Serial.print("Angle: ");
        Serial.println(ultrasonicAngle);
        unsigned long startTime = millis();

```

```
delay(100);  
while (millis() - startTime < 2000) {  
  scanForObstacles();  
  if(distance_in < 18){  
    return 0;  
  }  
  
}  
  
}
```

//Turn Left 90 degrees

```
void turnLeft(){  
  // Turn on motors  
  digitalWrite(in1, LOW);  
  digitalWrite(in2, HIGH);  
  digitalWrite(in3, LOW);  
  digitalWrite(in4, LOW);  
  
  analogWrite(enA, 255);  
  analogWrite(enB, 255);  
  
  delay(20);
```

// Speed analog values for motors, change for testing

```
int LeftSpeed = 150;
int RightSpeed = 0;
analogWrite(enA, LeftSpeed);
analogWrite(enB, RightSpeed);
```

```
delay(1000);
```

```
// Now turn off motors
```

```
digitalWrite(in1, LOW);
digitalWrite(in2, LOW);
digitalWrite(in3, LOW);
digitalWrite(in4, LOW);
```

```
delay(480);
```

```
}
```

```
//Function to drive continuously at speed
```

```
void driveContinuously(){
```

```
// Turn on motors
```

```
digitalWrite(in1, LOW);
digitalWrite(in2, HIGH);
digitalWrite(in3, HIGH);
digitalWrite(in4, LOW);
```

```
analogWrite(enA, 255);
```

```
analogWrite(enB, 255);
```

```
delay(20);
```

```
analogWrite(enA, 110);
```

```
analogWrite(enB, 100);
```

```
}
```

```
void driveBackwards(){
```

```
    // Turn on motors
```

```
    digitalWrite(in1, HIGH);
```

```
    digitalWrite(in2, LOW);
```

```
    digitalWrite(in3, LOW);
```

```
    digitalWrite(in4, HIGH);
```

```
    analogWrite(enA, 255);
```

```
    analogWrite(enB, 255);
```

```
    delay(20);
```

```
    // Speed analog values for motors, change for testing
```

```
    int LeftSpeed = 120;
```

```
    int RightSpeed = 100;
```

```
    analogWrite(enA, LeftSpeed);
```

```
    analogWrite(enB, RightSpeed);
```



```
delay(1000);
```

```
// Now turn off motors
```

```
digitalWrite(in1, LOW);
```

```
digitalWrite(in2, LOW);
```

```
digitalWrite(in3, LOW);
```

```
digitalWrite(in4, LOW);
```

```
delay(480);
```

```
}
```

```
//Function to stop driving
```

```
void stopDriving(){
```

```
    digitalWrite(in1, LOW);
```

```
    digitalWrite(in2, LOW);
```

```
    digitalWrite(in3, LOW);
```

```
    digitalWrite(in4, LOW);
```

```
    delay(500);
```

```
}
```

```
int getAverageDistance(int newReading) {
```

```
    // subtract the oldest reading
```

```
    total -= distanceReadings[readIndex];
```

```
    // store the new reading
```

```
    distanceReadings[readIndex] = newReading;
```

```
// add the new reading
```

```
total += newReading;
```

```
// move to next index
```

```
readIndex = (readIndex + 1) % NUM_READINGS;
```

```
// calculate average
```

```
averageDistance = total / NUM_READINGS;
```

```
return averageDistance;
```

```
}
```